

Plan qualité QP V0.4 (10-2026)

Ce document entend définir un plan qualité pour assurer la qualité des livrables du projet VibeUp.

Journal de version			
Version	Date	Auteur	Commentaire
0.1	02-02-2026	Gaël Brebel	
0.2	29-06-2026	Gaël Brebel	
0.3	02-07-2026	Gaël Brebel	Mise à jour OBS
0.3.1	17-09-2026	Gaël Brebel	Ajout de la spécification
0.4	09-10-2026	Gaël Brebel	Indication réorganisation des points de suivis entre équipes individuelles puis entre chefs d'équipes

Document maintenu par : PM

Prochaine révision planifiée : Après livraison du projet

Sommaire

Plan qualité QP V0.4 (10-2026).....	0
Journal de version.....	0
Sommaire.....	1
Rôles et responsabilités.....	3
Sayuri - Product Owner, développeuse backend.....	3
Timoty - DevOps, infra, assurance qualité.....	3
Gaël - Product manager, développeur web.....	3
Franck - Tech lead, designer, commercial, développeur web.....	3
Félix - Développeur mobile.....	3
Mathieu - Développeur mobile.....	3
Haifa - Développeuse back-end.....	3
OBS.....	3
Processus de développement.....	3
Méthode Agile/SCRUM.....	3
Cycles.....	4
Jalons.....	4
Keynote 1 - Début de la documentation et du développement.....	4
Conventions et bonnes pratiques.....	5
Conventions de code.....	5
Nommage.....	5
Structure des fichiers.....	5
Standards de documentation.....	5
Workflow.....	5
Validation des tâches.....	5
GitFlow.....	6
Git/Github.....	6
Stratégie de branches.....	6
Commits conventionnels.....	6
Nommage des branches.....	6
Stratégie de tests.....	6
Exécution des tests.....	6
Quand tester.....	6
Qui teste quoi.....	6
Objectifs qualité.....	7
Qualité du code.....	7
Objectifs de performance.....	7
Workflow de gestion des bugs.....	7
Stratégie de gestion des bugs.....	7
1. Signalement.....	7
2. Évaluation.....	8
3. Résolution.....	8
4. Définition des priorités.....	9
Intégration CI/CD.....	9

Workflow GitHub Actions.....	9
Outils.....	9
Gestion des tickets.....	9

Rôles et responsabilités

Sayuri - Product Owner, développeuse backend

Timoty - DevOps, infra, assurance qualité

Gaël - Product manager, développeur web

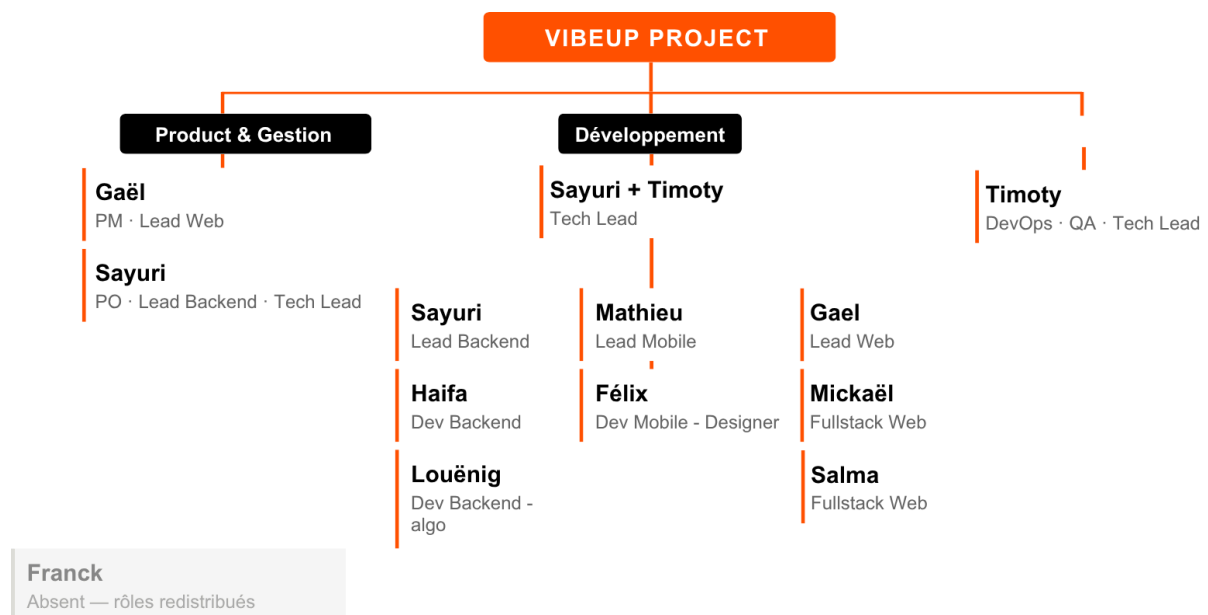
Franck - Tech lead, designer, commercial, développeur web

Félix - Développeur mobile

Mathieu - Développeur mobile

Haifa - Développeuse back-end

OBS



Processus de développement

Méthode Agile/SCRUM

- Pour s'assurer de la livraison du projet, comme nous ne pouvons pas moduler le temps ou les coûts de développement, nous choisissons la méthode Agile/SCRUM pour son adaptabilité et sa cohérence avec le travail asynchrone

- Nous définissons des jalons - un état du produit que nous souhaitons à des moments spécifiques (ex: les utilisateurs peuvent voir le feed mais pas la map, +2 semaines - les utilisateurs peuvent voir le feed et la map, +3 semaines - les comptes professionnels peuvent créer un compte et ajouter des événements eux même,...)
- Nous sommes répartis en plusieurs équipes :
 - Back
 - Mobile
 - Web

Chaque équipe a un chef d'équipe qui s'organise avec ses membres pour rédiger les tâches en ticket, se répartir les tâches et suivre l'avancement.

Lors des sprints planning/review, un point global est fait avec les chefs d'équipes et les autres membres de l'équipe s'ils sont disponibles

Ce changement d'organisation (plutôt que tout le temps tous ensemble), permet de faciliter l'organisation de réunions suite à l'augmentation de la taille de l'équipe.

Cycles

- 1 jours/semaine - école
 - Sprints - 4 semaines +
 - Réunions - 1 fois par semaine (~15-30min)
En visio si possible, par messages sinon (en asyncro)
 - Sprint review - à la fin du sprint
 - Sprint planning - à la suite de la sprint review
- 2 semaines d'affilées à l'école
 - Sprints - 2 semaines
 - Réunions - 1 fois par semaine (~15-30min)
En personne/visio ou par message
 - Sprint review - à la fin du sprint
 - Sprint planning - à la suite de la sprint review

Jalons

Keynote 1 - Début de la documentation et du développement

- L'organisation du projet est définie et la documentation associée est faite
- Un premier MVP répondant au premier jalon de l'évolution du produit est disponible

FU1 - 09-02-2026

- Une première itération de la documentation de l'organisation (incomplète mais avancée)
 - Plan qualité
 - Exigences fonctionnelles logicielles
 - Definition of Done
 - Besoins matériels
 - Description fonctionnelle logicielle

- Feuille de route

FU2 - 11-03-2026

- La documentation, si elle n'est pas finalisée, est proche de l'être
 - Feuille de route avec estimations budgétaires
 - Le développement a démarré (au moins quelques POCs)

FU3 - 06-05-2026

- La documentation est terminée (contenu et présentation)
- Le développement est en cours et montre de l'efficacité dans ses processus

Conventions et bonnes pratiques

Conventions de code

Nommage

cf : [SQC - Stratégie de qualité de code \(4. Standards de code\)](#)

Structure des fichiers

cf : [SQL - Stratégie de qualité de code \(5. Structure des fichiers\)](#)

Standards de documentation

Les documentations DOIVENT être écrites en français.

L'historique des versions de la documentation DOIT être mis à jour à chaque modification, en indiquant l'auteur, la date et le numéro de version de ladite modification.

Il faut mettre à jour l'en-tête avec les bonnes informations également, ainsi que la table des matières

Workflow

Validation des tâches

cf : [DOD - Definition of done](#)

GitFlow

Git/Github

Le processus de développement suit le workflow suivant :

1. **Création de branche** — Le développeur crée une branche de fonctionnalité (*feature branch*) directement depuis le ticket Jira associé, ce qui génère automatiquement la branche correspondante dans le dépôt GitHub.
2. **Développement local** — Le développeur travaille en local en veillant à maintenir sa branche synchronisée avec la branche *dev*.
3. **Commits réguliers** — Les modifications sont committées de manière régulière et incrémentale, à chaque étape fonctionnelle jugée satisfaisante.
4. **Mise à jour et vérification avant push** — Avant de pousser son travail sur la branche distante, le développeur met à jour sa branche locale par rapport à *dev*, puis exécute la suite de tests unitaires via Jest afin de s'assurer de la non-régression du code.
5. **Push sur la branche distante** — Une fois les tests validés, le développeur pousse ses modifications sur sa branche de fonctionnalité distante.
6. **Ouverture d'une Pull Request** — Le développeur ouvre une Pull Request (PR) de sa branche de fonctionnalité vers la branche *dev*. Un *squash* des commits est recommandé afin de maintenir un historique de version clair et lisible.
7. **Revue de code et validation** — La PR doit être revue et approuvée par au minimum deux autres membres de l'équipe avant de pouvoir être fusionnée (*merge*) dans la branche *dev*. Si les reviewers ne sont pas assignés automatiquement, c'est au développeur qui ouvre la PR de les déclarer dans sa PR.

Stratégie de branches

cf : [GN V1 - Git normage](#)

Commits conventionnels

cf : [GN V1 - Git normage](#)

Nommage des branches

cf : [GN V1 - Git normage](#)

Stratégie de tests

Exécution des tests

Quand tester

cf : [SQC - Stratégie de qualité de code \(6. Processus\)](#)

Qui teste quoi

cf : [SQC - Stratégie de qualité de code \(6. Processus - responsabilité\)](#)

Objectifs qualité

Qualité du code

cf : [SQC - Stratégie de qualité de code](#)

Objectifs de performance

cf : [SQC - Stratégie de qualité de code](#)

Workflow de gestion des bugs

Stratégie de gestion des bugs

1. Signalement

Lorsqu'un bug est identifié, le processus de signalement est le suivant :

- **Création d'un ticket** — Un ticket de type *Bug* est créé dans Jira, renseigné avec une description précise du problème, les étapes de reproduction, le comportement attendu et le comportement observé.
- **Notification de l'équipe** — Un message est envoyé dans le canal Microsoft Teams dédié au projet, en mentionnant le chef d'équipe et/ou le Tech Lead, afin d'assurer une prise en charge rapide.

2. Évaluation

La criticité du bug ainsi que la complexité de sa résolution sont évaluées conjointement par le(s) développeur(s) concerné(s) et le Tech Lead. Cette évaluation détermine le délai de traitement selon la matrice suivante :

Criticité	Définition	Délai de traitement
Critique	Le bug empêche le fonctionnement de l'application ou rend une fonctionnalité majeure inutilisable.	Prise en charge immédiate, indépendamment du sprint en cours.
Élevée	Le bug dégrade les performances ou l'expérience utilisateur sans bloquer le fonctionnement global.	Résolution durant le sprint en cours si la complexité est mineure à modérée ; reporté au sprint suivant dans le cas contraire.
Modérée	Le bug affecte l'interface utilisateur ou un aspect secondaire, sans impact fonctionnel significatif.	Planification au prochain sprint.
Faible	Le bug est cosmétique ou non perceptible par l'utilisateur final.	Planification au prochain sprint, selon la capacité disponible.

3. Résolution

Le bug est assigné au développeur ou au binôme compétent en fonction du périmètre impacté (mobile, backend ou web). Le DevOps/QA est sollicité pour valider la correction via les tests de non-régression avant la mise en production.

4. Définition des priorités

Priorité	Traitement
P1	Prise en charge immédiate. Le développeur concerné interrompt sa tâche en cours. Le Tech Lead et le PM sont notifiés.
P2	Intégré au sprint en cours. Traité en priorité dès qu'une tâche en cours est terminée.
P3	Planifié pour le sprint suivant lors du sprint planning
P4	Ajouté au backlog. Traité selon la capacité disponible de l'équipe

Intégration CI/CD

Workflow GitHub Actions

À définir

Outils

À définir

CI/CD : GitHub Actions

Gestion des tickets

Outil : Jira

Workflow : À définir

Labels et priorités

Attribution